

Knowledge Transfer Document: Speed Monitoring System

Overview

This project implements a Speed Monitoring System using a Raspberry Pi 4B. It integrates:

- TSR20 Nanoradar for real-time speed detection
- CP Plus IP Camera (RTSP stream) to capture overspeeding events
- 64x48 RGB P10 LED display for live visual feedback
- A Flask-based web application with REST API to set speed limits and view violations
- A virtual environment for dependency isolation and a systemd service to ensure the system starts on boot

Hardware Used

- Raspberry Pi 4B
- TSR20 Nanoradar (RS485 to USB via pyserial)
- CP Plus IP Camera (RTSP stream)
- 6x RGB P10 Panels (32x16, arranged 2 columns × 3 rows)
- 5V power supply for P10 LED panels
- Wiring connectors and optional relays

P10 Display Pin Configuration

HUB75 signal mapping to Raspberry Pi GPIO:

```
R1 -> GPIO 17
G1 -> GPIO 18
B1 -> GPIO 22
R2 -> GPIO 23
G2 -> GPIO 24
B2 -> GPIO 25
A -> GPIO 7
B -> GPIO 8
C -> GPIO 9
OE -> GPIO 2
CLK -> GPIO 3
LAT -> GPIO 4
GND -> GND
```

TSR20 Nanoradar Serial Port Detection

To detect the serial port the radar is connected to:

```
$ ls /dev/ttyUSB*
```

Software Environment Setup

1. Create Virtual Environment:

```
$ python3 -m venv myenv
```

```
$ source myenv/bin/activate
```

```
$ pip install opencv-python flask numpy pyserial waitress pillow
```

2. Clone and Build RGB LED Matrix Library:

```
$ git clone https://github.com/hzeller/rpi-rgb-led-matrix.git
```

```
$ cd rpi-rgb-led-matrix
```

Knowledge Transfer Document: Speed Monitoring System

```
$ make build-python PYTHON=$(which python3)
$ sudo make install-python
```

Flask Web Interface

The Flask web interface supports:

1. Speed Limit Input (via UI and API)
2. Viewing Screenshots of overspeeding events
3. REST API access to automate or remote control the system
4. Company branding via static/logo.png

Speed limit is stored in settings.txt as a single number (e.g., 60)

REST API Endpoints

Method	Endpoint	Description
GET	/get_speed_limit	Returns the current speed limit
POST	/set_speed_limit	Sets a new speed limit ({ "limit": 60 })
GET	/violations	Lists saved screenshots
GET	/violation/<name>	Returns a specific screenshot image

Screenshot Logic

If the detected speed exceeds the speed limit:

- A frame is captured from the RTSP camera stream using OpenCV
- Saved in a folder named datetime.now().strftime("%Y%m%d")
- Filenames: HHMMSS.jpg
- Example: /home/pi/20250621/153412.jpg

Autostart on Boot (systemd Service)

1. Create service file:

```
$ sudo nano /etc/systemd/system/speedmonitor.service
```

Add:

[Unit]

Description=Speed Monitoring System

After=network.target

[Service]

ExecStart=/home/pi/myenv/bin/python /home/pi/speedmonitor/final_code.py

WorkingDirectory=/home/pi/speedmonitor/

Restart=always

User=pi

Environment="PATH=/home/pi/myenv/bin"

[Install]

WantedBy=multi-user.target

2. Enable service:

```
$ sudo systemctl daemon-reexec
```

Knowledge Transfer Document: Speed Monitoring System

```
$ sudo systemctl enable speedmonitor
$ sudo systemctl start speedmonitor
```

Project Folder Structure

```
speedmonitor/
??? final_code.py
??? settings.txt
??? flask_web/
?   ??? app.py
?   ??? static/
?   ?   ??? logo.png
?   ??? templates/
?       ??? index.html
??? 20250621/
?   ??? 153412.jpg
??? myenv/
??? requirements.txt
```

Appendix: Full Source Code (final_code.py)

```
import cv2
from datetime import datetime
import os
import time
import threading
import serial
import subprocess
from rgbmatrix import RGBMatrix, RGBMatrixOptions, graphics
from PIL import Image, ImageDraw, ImageFont

# ===== Simulated Speeds =====
ser = serial.Serial('/dev/ttyUSB0', baudrate=9600, timeout=1)

# ===== Configurations =====
RTSP_URL =
'rtsp://admin:admin12345@192.168.1.250:554/cam/realmonitor?channel=1&subtype=0'
SETTINGS_FILE = "/home/pi/flask-webpage/settings.txt"

# ===== RGB Matrix Setup =====
options = RGBMatrixOptions()
options.rows = 16
options.cols = 32
options.chain_length = 6
options.parallel = 1
options.hardware_mapping = 'classic-pi1'
options.pwm_bits = 11
options.brightness = 80
options.show_refresh_rate = 0
options.scan_mode = 1
```

Knowledge Transfer Document: Speed Monitoring System

```
options.gpio_slowdown = 3
options.disable_hardware_pulsing = True

matrix = RGBMatrix(options=options)
canvas = matrix.CreateFrameCanvas()

# ===== Fonts & Colors =====
font = graphics.Font()
font.LoadFont("/home/pi/rpi-rgb-led-matrix/fonts/9x15.bdf")

font_small = graphics.Font()
font_small.LoadFont('/home/pi/rpi-rgb-led-matrix/fonts/6x9.bdf')

font_large = graphics.Font()
font_large.LoadFont('/home/pi/rpi-rgb-led-matrix/fonts/9x13.bdf')

color_safe = graphics.Color(0, 255, 0)
color_warn = graphics.Color(255, 165, 0)
color_alert = graphics.Color(255, 0, 0)

font_path = "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf"
font_size = 24
pil_font = ImageFont.truetype(font_path, font_size)

# ===== Panel Mapping =====
def get_panel_offset(panel_name):
    mapping = {
        'top_left': 0,
        'top_right': 32,
        'mid_left': 64,
        'mid_right': 96,
        'bot_left': 128,
        'bot_right': 160
    }
    return mapping[panel_name]

# ===== Draw Speed Split =====
def draw_large_split_text(canvas, text, color, top_left_x, top_left_y, mid_left_x,
mid_left_y):
    image = Image.new("RGB", (32, 32), "black")
    draw = ImageDraw.Draw(image)
    draw.text((0, 0), text, font=pil_font, fill=(color.red, color.green, color.blue))

    top_half = image.crop((0, 0, 32, 16))
    bottom_half = image.crop((0, 16, 32, 32))

    for y in range(top_half.size[1]):
        for x in range(top_half.size[0]):
            r, g, b = top_half.getpixel((x, y))
            canvas.SetPixel(top_left_x + x, top_left_y + y, r, g, b)
```

Knowledge Transfer Document: Speed Monitoring System

```
for y in range(bottom_half.size[1]):
    for x in range(bottom_half.size[0]):
        r, g, b = bottom_half.getpixel((x, y))
        canvas.SetPixel(mid_left_x + x, mid_left_y + y, r, g, b)

# ===== Helper Functions =====
def get_speed_limit():
    try:
        with open(SETTINGS_FILE, "r") as f:
            return int(f.read().strip())
    except Exception as e:
        print(f"Settings error: {e}")
        return 70

def update_display(speed, speed_limit):
    canvas.Clear()

    if speed <= speed_limit - 10:
        color = color_safe
        alert = "Safe Speed"
    elif speed_limit - 10 < speed <= speed_limit:
        color = color_warn
        alert = "Warning"
    else:
        color = color_alert
        alert = "SLOW DOWN!"

    speed_text = str(speed)
    draw_large_split_text(
        canvas,
        speed_text,
        color,
        get_panel_offset('top_left'),
        0,
        get_panel_offset('mid_left'),
        0
    )

    x_kmh = get_panel_offset('mid_right') + 2
    graphics.DrawText(canvas, font_small, x_kmh, 12, color, "km/h")

    alert_width = graphics.DrawText(canvas, font_small, 0, 0, graphics.Color(0, 0, 0),
    alert)
    x_alert = get_panel_offset('bot_left') + 64 - (alert_width // 2)
    graphics.DrawText(canvas, font_small, x_alert, 12, color, alert)

    matrix.SwapOnVSync(canvas)

def read_speed():
    data = ser.read(4)
    if len(data) < 4:
```

Knowledge Transfer Document: Speed Monitoring System

```
        return 0
    if data[0] == 0xFC and data[1] == 0xFA and data[3] == 0x00:
        return data[2]
    return 0

def overlay_speed_info(frame, radar_speed, speed_limit):
    text = f"Speed: {radar_speed} km/h | Limit: {speed_limit} km/h"
    font = cv2.FONT_HERSHEY_SIMPLEX
    font_scale = 0.8
    font_thickness = 2
    (text_width, text_height), baseline = cv2.getTextSize(text, font, font_scale,
font_thickness)
    x, y = 20, 40
    cv2.rectangle(frame, (x - 10, y - text_height - 10), (x + text_width + 10, y + 10),
(255, 255, 255), -1)
    cv2.putText(frame, text, (x, y), font, font_scale, (0, 0, 0), font_thickness)
    return frame

def capture_screenshot(speed, speed_limit):
    timestamp = datetime.now().strftime('%H%M%S')
    date_str = datetime.now().strftime('%Y%m%d')
    folder_name = f"/home/pi/{date_str}"
    os.makedirs(folder_name, exist_ok=True)
    filename = f"{timestamp}.jpg"
    save_path = os.path.join(folder_name, filename)

    try:
        cap = cv2.VideoCapture(RTSP_URL)
        if not cap.isOpened():
            print("? Failed to connect to camera")
            return

        start_time = time.time()
        ret = False
        while time.time() - start_time < 5:
            ret, frame = cap.read()
            if ret:
                break
            time.sleep(0.1)

        if not ret:
            print("? Frame capture failed")
            return

        annotated_frame = overlay_speed_info(frame, speed, speed_limit)
        cv2.imwrite(save_path, annotated_frame)
        print(f"? Saved violation: {save_path}")
    except Exception as e:
        print(f"?? Capture error: {e}")
    finally:
        if 'cap' in locals():
```

Knowledge Transfer Document: Speed Monitoring System

```
cap.release()

def record_hourly_stream():
    while True:
        current_time = datetime.now()
        date_str = current_time.strftime('%Y%m%d')
        hour_str = current_time.strftime('%H')
        folder_path = f"/home/pi/{date_str}/videos"
        os.makedirs(folder_path, exist_ok=True)

        output_filename = f"{date_str}_{hour_str}.mp4"
        output_path = os.path.join(folder_path, output_filename)

        ffmpeg_command = [
            'ffmpeg',
            '-rtsp_transport', 'tcp',
            '-i', RTSP_URL,
            '-t', '3600',
            '-vcodec', 'copy',
            '-acodec', 'copy',
            output_path
        ]

        print(f"?? Starting FFmpeg recording: {output_path}")
        try:
            subprocess.run(ffmpeg_command, check=True)
            print(f"? Finished recording: {output_path}")
        except subprocess.CalledProcessError as e:
            print(f"?? FFmpeg failed: {e}")
            time.sleep(5)

# ===== MAIN =====
if __name__ == "__main__":
    try:
        threading.Thread(target=record_hourly_stream, daemon=True).start()
        while True:
            speed = read_speed()
            speed_limit = get_speed_limit()

            update_display(speed, speed_limit)

            if speed > speed_limit:
                print(f"?? Violation: {speed} km/h")
                capture_screenshot(speed, speed_limit)
            else:
                print(f"? Safe speed: {speed} km/h")

            time.sleep(0.2)

    except KeyboardInterrupt:
        print("\n?? System stopped")
```

Knowledge Transfer Document: Speed Monitoring System

Autostart on Boot (All Components)

We'll create three `systemd` services so that each major component launches on boot and runs independently.

1. Speed Monitoring Service (final_code.py):

```
$ sudo nano /etc/systemd/system/speedmonitor.service
```

```
[Unit]
```

```
Description=Speed Monitoring System (final_code.py)
```

```
After=network.target
```

```
[Service]
```

```
ExecStart=/home/pi/myenv/bin/python /home/pi/speedmonitor/final_code.py
```

```
WorkingDirectory=/home/pi/speedmonitor/
```

```
Restart=always
```

```
User=pi
```

```
Environment="PATH=/home/pi/myenv/bin"
```

```
[Install]
```

```
WantedBy=multi-user.target
```

2. Flask Web App Service (app.py):

```
$ sudo nano /etc/systemd/system/flaskweb.service
```

```
[Unit]
```

```
Description=Flask Web UI for Speed Monitoring
```

```
After=network.target
```

```
[Service]
```

```
ExecStart=/home/pi/myenv/bin/python /home/pi/speedmonitor/flask_web/app.py
```

```
WorkingDirectory=/home/pi/speedmonitor/flask_web/
```

```
Restart=always
```

```
User=pi
```

```
Environment="PATH=/home/pi/myenv/bin"
```

```
[Install]
```

```
WantedBy=multi-user.target
```

3. REST API Service (restapi.py):

```
$ sudo nano /etc/systemd/system/restapi.service
```

```
[Unit]
```

```
Description=REST API Service for Speed Monitoring
```

```
After=network.target
```

```
[Service]
```

```
ExecStart=/home/pi/myenv/bin/python /home/pi/speedmonitor/flask_web/restapi.py
```

```
WorkingDirectory=/home/pi/speedmonitor/flask_web/
```

```
Restart=always
```

```
User=pi
```


Knowledge Transfer Document: Speed Monitoring System

Environment="PATH=/home/pi/myenv/bin"

[Install]

WantedBy=multi-user.target

To enable all services:

\$ sudo systemctl daemon-reexec

\$ sudo systemctl enable speedmonitor

\$ sudo systemctl enable flaskweb

\$ sudo systemctl enable restapi

\$ sudo systemctl start speedmonitor

\$ sudo systemctl start flaskweb

\$ sudo systemctl start restapi